DOCUMENTATION OFFICIELLE RÉVISÉE • CAMRAIL V2.0

Guide Universel de Déploiement & Manuel de Dépannage

Procédure complète, agnostique du poste de travail (Windows 10/11/Server), du clonage GitHub à la 1ère requête RAG.

Projet : RailMind Lite CAMRAIL **Environnement :** Windows Universel **Date :** Octobre 2026 **Statut :** 100% Opérationnel & Validé

🎯 1. Introduction & Objectif de la Documentation

Ce document constitue le **manuel technique officiel et exhaustif** de déploiement pour la solution **RailMind Lite (CAMRAIL)**. Contrairement aux guides initiaux comportant des chemins codés en dur ou omettant des dépendances système, ce manuel est **universel** : il s'adapte à n'importe quel poste de travail Windows (disque C: ou D:, versions variables de PostgreSQL, noms d'utilisateurs distincts).

Il détaille chaque composant (PostgreSQL 18+ & pgvector, backend FastAPI, moteur LLM souverain Ollama, frontend Next.js) et fournit la **solution exacte pour l'ensemble des pannes** rencontrées sur le terrain.

⚠️ 2. Matrice Comparative : Guide Initial vs Réalité & Solutions

Ce tableau récapitule les 7 divergences critiques constatées entre le guide d'origine et l'installation concrète :

Composant	Ce qu'indiquait le Guide initial	Problème / Bug réel rencontré	Solution Universelle Appliquée
1. Emplacement	Clonage libre (ex. Bureau / OneDrive)	Verrous d'écriture (`EBUSY`), lenteurs extrêmes, chemins corrompus.	Résolu Cloner impérativement hors OneDrive (ex: `C:\Projets\...` ou racine d'un disque).
2. PostgreSQL	Exécuter `CREATE EXTENSION vector;`	`ERROR: extension "vector" is not available` . Non inclus par défaut sous Windows.	Résolu Télécharger les binaires de pgvector et copier la DLL dans `lib\` et SQL dans `share\extension\` .
3. Fichier .env	`postgresql://user:pass@host:5432/db`	Crash du parseur SQLAlchemy / Alembic si le mot de passe contient le caractère `@` .	Résolu Règle RFC 3986 : Remplacer `@` par `%40` dans la chaîne de connexion.
4. Python venv	`pip install -r requirements.txt`	`ModuleNotFoundError: No module named 'authlib'` (ou `ollama` , `itsdangerous`).	Résolu Installer manuellement : `pip install authlib itsdangerous ollama pytz` .
5. Embeddings	Téléchargement transparent automatique	Blocage de `seed_demo.py` (>30 min) ou crash sur valeurs NaN (coupure réseau).	Résolu Nettoyer le cache safetensors et sécuriser `app/services/indexing.py` avec `np.nan_to_num` .
6. Routage API	`NEXT_PUBLIC_API_URL=http://localhost:8000/api/v1`	Bandeau rouge **"Not Found"** sur le bouton de connexion (404).	Résolu Régler l'URL sur `http://localhost:8000` et ajouter le double routage (`/auth` et `/api/v1/auth`) dans `main.py` .
7. Cache Next.js	Modifier `.env.local` et tester	Le message d'erreur persiste car les variables d'environnement sont figées au build.	Résolu Arrêter le serveur frontend (`Ctrl + C`) et relancer systématiquement `npm run dev` .

📋 3. Conventions & Table des Variables Universelles

Pour rendre les commandes exécutables sur **n'importe quel PC**, ce document utilise des variables entre crochets `<...>` :

Variable	Description	Exemples courants
`<DOSSIER_PROJET>`	Chemin complet où le code source est situé	`C:\Projets\camrail` ou `D:\camrail-main`
`<CHEMIN_POSTGRES>`	Dossier d'installation de PostgreSQL	`C:\Program Files\PostgreSQL` ou `D:\PostgreSQL_Serveur`
`<PG_VERSION>`	Numéro de version majeure de PostgreSQL	`15`, `16`, `17` ou `18`
`<DB_USER>`	Utilisateur administrateur PostgreSQL	`postgres`
`<DB_PASS>`	Mot de passe de la base de données	Votre mot de passe local
`<DB_PORT>`	Port réseau d'écoute de la base	`5432`
`<DB_NAME>`	Nom de la base de données de l'application	`railmind`
`<DOSSIER_FRONTEND>`	Sous-dossier contenant l'application Next.js	`frontend\camrail-rda` ou `frontend`

⚙️ 4. Règles Fondamentales d'Environnement Système

> 🚨 **RÈGLE N°1 : BANNIR STRICTEMENT ONEDRIVE, DROPBOX & CLOUD SYNC**
> Les dossiers `Desktop` et `Documents` sous Windows sont souvent synchronisés automatiquement par OneDrive. Les dossiers `node_modules` et `venv` génèrent des dizaines de milliers de fichiers. La synchronisation continue crée des verrous de fichiers (erreurs `EBUSY` ou `Permission denied`) et détruit les environnements virtuels. **Placez toujours votre projet dans un dossier local pur (ex: C:\Projets\camrail).**

> 💡 **RÈGLE N°2 : NE JAMAIS COPIER UN DOSSIER VENV D'UNE AUTRE MACHINE**
> Un environnement virtuel Python enregistre les chemins absolus vers l'interpréteur de la machine d'origine. Si vous copiez un projet avec un dossier `venv` déjà présent, supprimez-le et recréez-le sur votre machine cible avec `python -m venv venv`.

🐘 5. Étape 1 : PostgreSQL & Extension pgvector (Indépendant de la version)

A. Création de la base de données

Ouvrez votre console SQL (pgAdmin 4, DBeaver ou invite `psql`) :

```
CREATE DATABASE railmind;
```

B. Installation manuelle de pgvector sur Windows

L'installateur standard de PostgreSQL Windows ne contient pas `pgvector`. Suivez cette méthode universelle :

1. Rendez-vous sur les releases officielles du projet GitHub : `pgvector/pgvector/releases`.
2. Téléchargez l'archive binaire Windows correspondant à votre version `<PG_VERSION>`.
3. Copiez les fichiers extraits dans votre dossier `<CHEMIN_POSTGRES>` :
   - Fichier `vector.dll` ➡️ Dans `<CHEMIN_POSTGRES>\lib\`
   - Fichiers `vector.control` et `vector--*.sql` ➡️ Dans `<CHEMIN_POSTGRES>\share\extension\`
4. Connectez-vous à la base `railmind` et activez l'extension :

```
\c railmind;
CREATE EXTENSION vector;
```

5. **Vérification du succès :**

```
SELECT extname, extversion FROM pg_extension WHERE extname = 'vector';
```

La table doit afficher la ligne 'vector' avec sa version.

6. Étape 2 : Backend FastAPI & Pipeline RAG

A. Création de l'environnement virtuel

```
cd "<DOSSIER_PROJET>\backend"
python -m venv venv
.\venv\Scripts\Activate.ps1
```

(Si PowerShell bloque l'exécution des scripts : `Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned`)

B. Installation des bibliothèques nécessaires

```
python -m pip install --upgrade pip
pip install -r requirements.txt
pip install authlib itsdangerous ollama pytz
```

C. Configuration du fichier `backend/.env`

Créez ou modifiez le fichier `.env` dans `<DOSSIER_PROJET>\backend` :

```
# Base de données PostgreSQL
# RAPPEL : Encoder les caractères spéciaux du mot de passe (@ → %40, # → %23)
DATABASE_URL=postgresql://<DB_USER>:<DB_PASS_ENCODÉ>@localhost:<DB_PORT>/railmind

# Sécurité JWT (chaîne aléatoire longue)
JWT_SECRET=cle_secrete_super_robuste_camrail_2026_production
JWT_ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=480

# Modèle d'embeddings & LLM
EMBEDDING_MODEL_NAME=sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_MODEL=llama3.2:3b

# Chemins et logs
UPLOAD_DIR=./uploads
LOG_LEVEL=INFO
```

> ⚠️ **ENCODAGE OBLIGATOIRE DES CARACTÈRES SPÉCIAUX DANS LE MOT DE PASSE**
>
> Si votre mot de passe contient le caractère `@` (ex: `MonPass@123`), il casse le parseur URL. Vous **DEVEZ** le remplacer par `%40` dans `DATABASE_URL` (ex: `MonPass%40123`). De même, remplacez `#` par `%23` et `%` par `%25`.

D. Exécution des migrations Alembic

```
python -m alembic upgrade head
```

Toutes les tables (utilisateurs, documents, fragments vectoriels) sont générées dans PostgreSQL.

E. Blindage du code contre les tenseurs corrompus (NaN)

Dans `<DOSSIER_PROJET>\backend\app\services\indexing.py`, garantissez l'assainissement des vecteurs :

```
import numpy as np

# Remplacement de tout NaN ou Inf par une valeur valide
embedding = np.nan_to_num(embedding, nan=0.0, posinf=1.0, neginf=-1.0).tolist()
```

F. Injection des données de démonstration (`seed_demo.py`)

```
python seed_demo.py
```

Le script crée 3 comptes prédéfinis et indexe les documents ferroviaires CAMRAIL :

- **Administrateur :** `admin@camrail.net` / `adminpassword`
- **Gestionnaire GED :** `docadmin@camrail.net` / `docadminpassword`
- **Consultation seule :** `readonly@camrail.net` / `readonlypassword`

🦙 7. Étape 3 : Moteur LLM Souverain (Ollama)

1. Téléchargez et installez **Ollama** pour Windows (`https://ollama.com`).
2. Ouvrez un terminal et téléchargez le modèle optimisé :

```
ollama pull llama3.2:3b
```

3. Vérifiez qu'Ollama répond sur `http://localhost:11434/` (doit afficher *"Ollama is running"*).

⚛️ 8. Étape 4 : Frontend Next.js & Alignement des URLs

A. Configuration du fichier `.env.local`

Dans `<DOSSIER_PROJET>\<DOSSIER_FRONTEND>\.env.local` :

```
# Point d'accès de l'API Backend
NEXT_PUBLIC_API_URL=http://localhost:8000
```

> ⚠️ **POURQUOI NE PAS METTRE /api/v1 DANS NEXT_PUBLIC_API_URL ?**
> Le code source du frontend construit ses requêtes sous la forme `${NEXT_PUBLIC_API_URL}/auth/login`. Si vous mettez `http://localhost:8000/api/v1`, la requête envoyée sera `http://localhost:8000/api/v1/auth/login`, ce qui déclenchait la fameuse erreur **404 Not Found** lors du clic sur le bouton de connexion !

B. Sécurisation Backend pour double compatibilité (`main.py`)

Dans `backend▯pp\main.py`, les routes acceptent les deux formats :

```
# Préfixes directs standards
app.include_router(auth.router, prefix="/auth", tags=["auth"])
app.include_router(documents.router, prefix="/documents", tags=["documents"])
app.include_router(assistant.router, prefix="/assistant", tags=["assistant"])

# Préfixes de compatibilité v1
app.include_router(auth.router, prefix="/api/v1/auth", tags=["auth-v1"])
app.include_router(documents.router, prefix="/api/v1/documents", tags=["documents-v1"])
app.include_router(assistant.router, prefix="/api/v1/assistant", tags=["assistant-v1"])
```

C. Installation et Démarrage du Frontend

```
cd "<DOSSIER_PROJET>\<DOSSIER_FRONTEND>"
npm install
npm run dev
```

Rappel : Toute modification du fichier `.env.local` exige d'arrêter le serveur (Ctrl + C) et de relancer `npm run dev`.

🚀 9. Étape 5 : Test Opérationnel Complet (De la Connexion à la Première Question)

A. Ordre de lancement des 4 services

1. **Service Ollama** : Actif en tâche de fond (Port 11434).
2. **PostgreSQL** : Service Windows démarré (Port 5432).
3. **Backend FastAPI** (Terminal 1) :

   ```
   cd "<DOSSIER_PROJET>▯ackend"
   .▯env\Scripts\Activate.ps1
   uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
   ```

4. **Frontend Next.js** (Terminal 2) :

   ```
   cd "<DOSSIER_PROJET>\<DOSSIER_FRONTEND>"
   npm run dev
   ```

B. Test de Connexion Utilisateur

1. Ouvrez votre navigateur sur : `http://localhost:3000/login`
2. Saisissez l'identifiant : `docadmin@camrail.net` et le mot de passe : `docadminpassword`
3. Cliquez sur **Se connecter**.

4. **Résultat validé :** Vous êtes instantanément connecté et redirigé vers le tableau de bord sans aucune erreur.

C. Test de la Première Question RAG (Chatbot Documentaire)

1. Cliquez sur l'onglet **Assistant** dans le menu de gauche.
2. Posez une question métier, par exemple :
   « Quelles sont les consignes de sécurité obligatoires lors des manœuvres en gare de triage ? »
3. **Validation du système :**
   - PostgreSQL effectue la recherche vectorielle des passages pertinents.
   - Le modèle Ollama génère une réponse claire et structurée en français.
   - L'interface affiche les **sources et références exactes des documents CAMRAIL** consultés.

🛠️ 10. Matrice Complète de Dépannage Rapide (Troubleshooting)

Erreur constatée	Cause technique	Action corrective
`ERROR: extension "vector" is not available`	Les fichiers binaires de pgvector ne sont pas présents dans les dossiers PostgreSQL.	Copier `vector.dll` dans `<CHEMIN_POSTGRES>\lib\` et les scripts `.sql` / `.control` dans `<CHEMIN_POSTGRES>\share\extension\`.
`could not translate host name` **ou crash Alembic**	Caractère `@` dans le mot de passe cassant la décomposition de `DATABASE_URL`.	Remplacer `@` par `%40` dans la chaîne `DATABASE_URL` de `backend/.env`.
Bandeau rouge `Not Found` **lors du login**	Le frontend appelle `/api/v1/auth/login` alors que le backend expose `/auth/login`.	1. Mettre `NEXT_PUBLIC_API_URL=http://localhost:8000` dans `.env.local`. 2. Redémarrer `npm run dev`.
`ModuleNotFoundError: No module named 'authlib'`	Dépendances manquantes dans le fichier `requirements.txt` initial.	Activer le venv et exécuter : `pip install authlib itsdangerous ollama pytz`.
`seed_demo.py` **bloqué ou crash sur NaN**	Coupure réseau lors du téléchargement des poids Hugging Face MiniLM.	Ajouter `np.nan_to_num()` dans `indexing.py` et relancer le script.
`Connection refused` **lors d'une question**	Le serveur Ollama n'est pas démarré sur le port 11434.	Lancer l'application Ollama ou exécuter `ollama serve` dans un terminal.
`Execution of scripts is disabled` **(PowerShell)**	Stratégie de sécurité Windows bloquant l'activation du script `Activate.ps1`.	Lancer dans la fenêtre : `Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned`.

⚡ 11. Aide-Mémoire des Commandes Fréquentes

Action	Commande PowerShell
Démarrer le Backend	`cd "<DOSSIER_PROJET>\backend"; .\venv\Scripts\Activate.ps1; uvicorn app.main:app --port 8000 --reload`
Démarrer le Frontend	`cd "<DOSSIER_PROJET>\<DOSSIER_FRONTEND>"; npm run dev`
Réinitialiser les données de démo	`cd "<DOSSIER_PROJET>\backend"; .\venv\Scripts\Activate.ps1; python seed_demo.py`
Vérifier le statut d'Ollama	`Invoke-RestMethod -Uri "http://localhost:11434/"`
Tester l'API Login en ligne de commande	`Invoke-RestMethod -Method Post -Uri "http://localhost:8000/auth/login" -Body @{username='docadmin@camrail.net';password='docadminpassword'} -ContentType 'application/x-www-form-urlencoded'`